

De l'Image Satellitaire au Modèle YOLOv8 : Une Architecture Microservice pour la Surveillance Maritime avec Sentinel-1

Ce rapport présente une analyse technique approfondie visant à guider l'intégration de données satellitaires Sentinel-1 dans le pipeline existant de surveillance maritime. Il aborde les quatre axes critiques définis pour la Phase II du projet "maritime-edge-ai", en se basant exclusivement sur les documents fournis. L'objectif est de fournir une feuille de route claire, évaluant la faisabilité, identifiant les risques et proposant des solutions techniques concrètes pour transformer le démonstrateur technologique en un système opérationnel robuste.

Axe 1 : Chaîne de Prétraitement des Images Sentinel-1 GRD pour la Compatibilité IA

L'intégration d'images Sentinel-1 Ground Range Detected (GRD) dans le pipeline existant, conçu pour des données SAR simulées, représente le principal risque technique du projet [23](#). La conversion de données radar brutes en un format exploitable par un modèle de vision par ordinateur comme YOLOv8n nécessite une chaîne de prétraitement rigoureuse et bien comprise. La faisabilité de cette étape est jugée **bloquante**, car elle dépend de choix algorithmiques qui n'ont pas été validés expérimentalement et dont les impacts sur la performance du modèle ne peuvent être qu'estimés sans tests ciblés. La réussite de cette phase est conditionnée par une compréhension précise des caractéristiques des données Sentinel-1 et par le développement d'un pipeline de traitement performant et compatible.

La première étape consiste à comprendre la structure d'un produit Sentinel-1 GRD téléchargé depuis le Copernicus Data Space Ecosystem (CDSE). Un tel produit est encapsulé dans un dossier `.SAFE` complexe [5](#). Ce dossier contient plusieurs éléments

essentiels pour le traitement. Le fichier `manifest.safe` contient les métadonnées principales du produit [43](#). Les données d'image elles-mêmes sont stockées dans des fichiers GeoTIFF, un par canal de polarisation. Pour le mode Interférométrique Large-Feuille (IW), qui est le plus adapté pour la surveillance maritime, les deux polarisations principales sont VV (Vertical-Vertical) et VH (Vertical-Horizontal) [16](#). La combinaison de ces deux canaux est souvent supérieure à l'utilisation d'une seule polarisation pour la détection de navires, car elle fournit des informations complémentaires sur la surface de la mer et les objets flottants [83](#). Les pixels dans ces GeoTIFFs sont typiquement codés sur 16 bits non signés (`uint16`), mais les valeurs brutes ne représentent pas directement le coefficient de réflexion radar. La dynamique de ces valeurs varie fortement et doit être traitée pour être exploitée par un réseau neuronal [7](#).

Le cœur du prétraitement réside dans la transformation de ces données brutes en une image d'intensité linéaire et normée. La première étape critique est la **calibration radiométrique**, qui vise à convertir les valeurs numériques brutes en coefficient de réflexion radar normalisé, souvent désigné par σ^0 (sigma-zéro) [6](#). Cette étape est **obligatoire** pour la détection de navires. Elle permet de compenser l'effet de l'angle d'incidence du radar, qui affecte fortement l'intensité réfléchie, et de rendre les valeurs comparables entre différentes images et passages du satellite [89](#) [106](#). Sans cette calibration, le modèle pourrait apprendre des artefacts liés à l'ombre géométrique plutôt que la présence physique d'un navire. L'étape suivante est le **filtrage du speckle**. L'imagerie SAR est intrinsèquement affectée par un bruit granulaire appelé speckle, qui résulte de la nature cohérente du processus radar [23](#). Si ce bruit n'est pas atténué, il peut être interprété par le réseau neuronal comme des petits objets, augmentant considérablement le nombre de faux positifs. Des filtres adaptatifs comme le filtre de Lee sont particulièrement efficaces car ils suppriment le speckle tout en préservant les bords et les textures des vrais objets, comme les contours des navires [85](#). Bien que certains modèles d'IA soient conçus pour être intrinsèquement robustes au speckle, son filtration préalable reste une pratique standard et fortement recommandée pour améliorer la qualité des entrées du modèle [81](#).

Après calibration et filtrage, les données doivent être **normalisées**. La distribution d'intensité d'une image SAR après traitement a une dynamique très large. Pour l'alimenter à un modèle YOLOv8, qui attend généralement des entrées normalisées sur un intervalle spécifique (par exemple, $[0, 1]$ ou $[0, 255]$), une étape de normalisation est indispensable. Une approche courante consiste à appliquer une fonction logarithmique aux valeurs σ^0 pour compresser la dynamique et simuler la perception humaine de l'intensité. Ensuite, une normalisation linéaire peut être appliquée, soit par rapport à un maximum global (fixe ou variable), soit par rapport au maximum observé sur chaque

patch local pour s'adapter aux variations de la scène [81](#) . Le choix de la méthode de normalisation est crucial et doit être aligné sur celle utilisée lors de l'entraînement initial du modèle sur les jeux de données SSDD et iVision-MRSSD. Cependant, aucune information sur le prétraitement spécifique de ces datasets n'est disponible dans les sources fournies, ce qui constitue un point d'incertitude majeur. La divergence de distribution entre les données Sentinel-1 et les données d'entraînement pourrait impacter significativement la performance du modèle. Par conséquent, une validation expérimentale est impérative pour trouver la chaîne de normalisation la plus appropriée. La dernière étape du prétraitement est le **découpage en tuiles**. Les modèles YOLO traitent des images de dimension fixe. Il faut donc découper la scène Sentinel-1 étendue en tuiles de la même taille que celles utilisées pendant l'entraînement. Pour éviter de couper des navires en deux, une stratégie de recouvrement (*overlap*) entre les tuiles adjacentes est nécessaire. La taille du recouvrement doit être suffisante pour capturer la longueur maximale attendue d'un navire dans la scène.

Pour mettre en œuvre ce prétraitement dans un environnement Docker léger, des librairies Python pures sont requises. `rasterio` est la librairie idéale pour lire les fichiers GeoTIFF et accéder aux données brutes et aux métadonnées associées [110](#). Pour les opérations de calibration et de filtrage, `pyroSAR` est une option puissante, car elle encapsule des algorithmes développés par l'ESA et facilite l'accès aux produits auxiliaires comme les fichiers d'orbite [5](#) . Toutefois, elle peut introduire des dépendances supplémentaires. Une alternative est de développer des fonctions personnalisées en utilisant `numpy` pour les calculs matriciels et `scipy.ndimage` pour les filtres. Des projets open-source sur GitHub, bien que rares, montrent des implémentations de pipelines complets qui peuvent servir de base, notamment ceux qui intègrent des techniques avancées comme la fusion multi-polarisation ou les algorithmes de détection CFAR (Constant False Alarm Rate) [16](#) [84](#) [88](#) . Ces références sont précieuses pour construire une solution robuste.

Étape de Prétraitement	Faisabilité	Incertitudes Clés
Structure & Caractéristiques	Facile	Aucune
Calibration Radiométrique	Modérée	Choix de la formule exacte (σ^0 , γ^0). Nécessite les fichiers d'orbite.
Filtrage du Speckle	Modérée	Sélection du filtre optimal (Lee, Frost, etc.) et réglage des paramètres.
Normalisation	Complexe	Point critique. Distribution d'intensité inconnue des datasets d'entraînement (SSDD/iVision-MRSSD). Validation expérimentale indispensable.
Découpage en Tuiles	Facile	Taille de la tuile, recouvrement.

En conclusion, si la mise en place des étapes de prétraitement individuelles est réalisable, leur intégration en une chaîne cohérente et compatible avec le modèle YOLOv8 reste un défi majeur. La principale barrière est l'absence de documentation sur le prétraitement des données d'entraînement. La validation expérimentale, en mesurant la performance du modèle sur des tuiles générées par différentes chaînes de traitement, sera le facteur déterminant du succès de cette phase.

Axe 2 : Comparaison Technique et Économique des Plateformes CDSE et Sentinel Hub

La sélection de la plateforme pour la recherche et le téléchargement des données Sentinel-1 est une décision stratégique qui impactera à la fois la performance, la flexibilité et le coût du système. Deux options majeures se présentent : le Copernicus Data Space Ecosystem (CDSE) et Sentinel Hub. Chacune offre un ensemble d'API et de services, mais leurs modèles respectifs posent des défis et offrent des opportunités distinctes. La faisabilité d'utiliser le CDSE est jugée **modérée** en raison de la nécessité de traiter des volumes de données importants localement, tandis que l'utilisation de Sentinel Hub est **facile** grâce à ses capacités de traitement en ligne, mais peut entraîner des coûts variables. Une approche hybride est probablement la plus judicieuse.

Le CDSE est un écosystème ouvert qui fournit un accès gratuit et instantané à une large gamme de données Copernicus ³. Pour automatiser la recherche de produits Sentinel-1 GRD IW, l'API OData est le principal outil. L'endpoint de recherche permet de formuler des requêtes complexes en spécifiant une boîte englobante géographique (bounding box), une plage de dates, ainsi que des filtres sur le type de produit (GRD), le mode (IW), la polarisation (VV/VH) et la direction d'orbite (ascendant/descendant) ^{98 102}. La réponse de l'API est un flux JSON structuré, contenant pour chaque produit trouvé un ID, son empreinte au sol (footprint), sa date de prise, sa taille, et surtout, une URL pour le téléchargement ¹⁰³. L'authentification pour un compte public se fait via un jeton Keycloak obtenu en utilisant le mécanisme OAuth2 avec `grant_type=password`. Bien que cette méthode soit stable, il est toujours prudent de vérifier sa validité dans la documentation officielle en vigueur en 2025. Une fois l'URL obtenue, le téléchargement se fait via un autre endpoint spécifique, généralement de la forme `https://zipper.dataspace.copernicus.eu/odata/v1/Products({id})/$value` ⁹⁹. Le volume des données est un point important à considérer : un produit GRD IW complet peut atteindre plusieurs gigaoctets (typiquement 1-3 Go). Bien que cela soit

techniquement viable pour un microservice doté d'un disque adéquat et d'une bonne bande passante, cela impose une gestion rigoureuse de l'espace de stockage et peut introduire une latence significative si la connexion Internet est lente. Une limitation majeure du CDSE est qu'il agit principalement comme un dépôt de données brutes. S'il est possible de télécharger une zone géographique précise, cette opération se fait après le téléchargement complet de la scène, ce qui peut être inefficace si la zone d'intérêt est petite par rapport à la taille totale de la scène. Le quota de téléchargement pour un compte public est également une contrainte potentielle à surveiller.

Sentinel Hub, quant à lui, propose un modèle différent centré sur le traitement des données directement dans le nuage. Il dispose d'un plan gratuit avec un nombre limité d'Unités de Traitement (PU) mensuelles, qui sont des crédits nécessaires pour exécuter des analyses sur les données satellites ¹. Son atout majeur est l'API de Traitement, qui permet de définir un script JavaScript (un *evalscript*) pour traiter les données. Grâce à cette API, il est possible de réaliser l'ensemble de la chaîne de prétraitement (calibration, filtrage du speckle, normalisation) directement sur le serveur de Sentinel Hub, puis de télécharger uniquement le résultat final, et potentiellement seulement la partie de l'image correspondant à une zone d'intérêt précise. Cette capacité à télécharger un sous-ensemble spatial extrait et prétraité est une avancée fondamentale. Elle permet de réduire drastiquement la quantité de données à transférer vers le microservice de détection, minimisant ainsi la latence réseau et la charge de travail locale. Cela répond directement au besoin d'une architecture légère et performante. Cependant, chaque exécution de script consomme des PUs. Pour un flux de données continu, la consommation de PUs pourrait rapidement devenir élevée, et le coût potentiel pourrait dépasser celui d'un simple téléchargement de données brutes, rendant la gestion budgétaire cruciale.

La comparaison entre les deux plateformes met en lumière leurs forces et faiblesses respectives. Le CDSE est une source de données brute fiable et gratuite, idéale pour la recherche initiale et le téléchargement de scènes entières pour des besoins analytiques ponctuels. Sentinel Hub excelle dans les scénarios où le traitement en amont et le téléchargement sélectif sont essentiels pour optimiser les ressources. Pour le projet maritime-edge-ai, une approche hybride semble être la stratégie la plus rationnelle : 1. Utiliser l'API OData du CDSE pour la phase de recherche et de catalogage des produits Sentinel-1 disponibles. 2. Pour le téléchargement des données d'entrée pour le modèle de détection, privilégier l'API de Traitement de Sentinel Hub. En définissant un *evalscript* adéquat, on peut extraire et prétraiter la zone d'intérêt directement dans le nuage, envoyant ainsi au microservice une image de détection propre et prête à l'emploi. 3. Conserver le CDSE comme source de repli lorsque les PUs de Sentinel Hub sont épuisés ou pour des cas d'usage hors ligne.

Cette approche combine la fiabilité du CDSE avec l'efficacité du traitement en ligne de Sentinel Hub, créant un pipeline flexible et optimisé.

Caractéristique	Copernicus Data Space Ecosystem (CDSE)	Sentinel Hub
Accès Gratuit	Oui, avec authentification par compte public 3	Oui, avec limite de Processing Units (PU) 1
API Principale	OData API pour la recherche 102	OData API pour la recherche + Process API pour le traitement 1
Authentification	Jeton Keycloak (OAuth2) 103	Clé d'API
Traitement en Amont	Non, données brutes	Oui, via <i>evalscript</i> JavaScript 1
Téléchargement Sélectif	Non, téléchargement de la scène complète ou via repliage	Oui, extraction d'une zone d'intérêt après traitement 1
Taille Typique Téléchargement	1-3 Go par produit GRD IW 99	Variable, dépend du crop et du traitement
Coût Principal	Bande passante, espace disque	Consommation d'Unités de Traitement (PU) 1
Faisabilité pour le Projet	Modérée (gestion de gros volumes)	Facile (optimisation de la latence)

En somme, bien que le CDSE soit une plateforme solide, les capacités uniques de la Process API de Sentinel Hub en font l'outil le plus pertinent pour l'architecture souhaitée, malgré le coût potentiel lié à la consommation de PU. La stratégie hybride est la voie à privilégier pour équilibrer flexibilité, performance et coût.

Axe 3 : Intégration de Données Orbitales et de Télémétrie via SatNOGS

L'enrichissement du pipeline de détection maritime avec des données orbitales et de santé du satellite est essentiel pour contextualiser les résultats et améliorer la fiabilité des alertes. L'intégration des données issues du Satellite Tracking and Orbit Determination Network (SatNOGS) est une démarche de faisabilité **facile à modérée**, car les API publiques pour récupérer les TLE (Two-Line Elements) et la télémétrie sont bien documentées et accessibles. Cette intégration transforme le système d'une simple détection en une station de contrôle de satellite capable d'analyser les conditions sous lesquelles les données ont été collectées.

La première composante de l'enrichissement est l'obtention des TLE. Les TLE sont des données standards utilisées pour propager l'orbite d'un satellite et prédire sa position future [19](#). Le réseau SatNOGS maintient une base de données publique et régulièrement

mise à jour de ces éléments orbitaux [67](#). L'API de SatNOGS expose un endpoint spécifique, <https://network.satnogs.org/api/tle/>, qui permet de récupérer les TLE pour un satellite donné, identifié par son nom ou son numéro NORAD [91](#). La requête retourne un format JSON qui contient les deux lignes TLE, prêt à être utilisé par des bibliothèques de propagation orbitale. La fréquence de mise à jour de ces TLE est suffisante pour la plupart des applications de simulation, mais pour des prédictions très précises sur le long terme, il est recommandé de les rafraîchir périodiquement (par exemple, toutes les 24 heures). Une fois les TLE récupérées, leur propagation pour obtenir la position et la vitesse du satellite à un instant précis peut être réalisée en utilisant des bibliothèques Python spécialisées. Des bibliothèques comme `satellite-js` [82](#) ou `skyfield` [19](#) implémentent les modèles de propagation orbitale (comme SGP4) nécessaires pour convertir les TLE en coordonnées géospatiales précises. L'intégration de cette logique dans le microservice de prétraitement permettra de connaître la position exacte du satellite au moment de la prise de vue de chaque image, un renseignement géolocalisant précieux.

Au-delà des données orbitales, la télémétrie SatNOGS apporte un éclairage sur l'état de santé et le fonctionnement du CubeSat. L'API de SatNOGS permet de récupérer des paquets de télémétrie pour un satellite et une période donnés [91](#). Ces paquets contiennent des informations cruciales qui peuvent être corrélées avec les détections d'objets. On y trouve des métriques telles que la température interne de l'On-Board Computer (OBC), le niveau de charge des batteries, la puissance générée par les panneaux solaires, et l'état des différents systèmes embarqués [48](#) [92](#). Ces données, souvent déjà décodées par SatNOGS, permettent d'évaluer la fiabilité d'une détection. Par exemple, une détection de navire effectuée alors que la batterie du satellite était à court ou que le système de communication était en mode veille pourrait être marquée comme moins fiable. L'intégration de ces données dans le pipeline consiste à enrichir chaque événement de détection généré par le modèle IA. Lorsqu'un événement est créé, le service de détection doit appeler un service de supervision du satellite pour récupérer la télémétrie la plus proche temporellement et l'inclure dans le payload de l'événement. Cela crée un lien direct et contextuel entre une observation (la détection) et l'état de la plateforme qui l'a produite.

Des exemples concrets de projets universitaires et open-source confirment la faisabilité de cette approche. Des travaux universitaires, comme ceux présentés lors du Latin American CubeSat Workshop, illustrent l'utilisation de données de suivi pour l'opérationnalisation de constellations de petits satellites [12](#). Bien que les détails spécifiques à chaque projet ne soient pas toujours disponibles, la méthodologie générale est bien établie : récupération des TLE et de la télémétrie via les APIs publiques, puis utilisation de ces données pour la

navigation, la planification et l'analyse des performances. La bibliothèque `satnogs-client` est explicitement mentionnée comme étant le logiciel responsable de l'automatisation de la réception des signaux et de la récupération des données auprès de la base de données SatNOGS [67](#). Son code source et sa documentation peuvent servir de référence pour implémenter la partie client de l'intégration.

L'implémentation technique requiert la création d'un nouveau microservice, par exemple nommé `satellite-monitor`, dont la seule responsabilité serait de gérer les interactions avec l'API SatNOGS. Ce service communiquerait avec les autres services (par exemple, `preprocessor` ou `detector`) via une API REST ou un message. La chaîne de traitement serait modifiée comme suit : après avoir reçu une image et ses métadonnées, le service de prétraitement n'appellerait pas seulement le modèle IA, mais aussi le service `satellite-monitor` pour obtenir la position du satellite et la télémétrie correspondantes. Les informations acquises seraient ensuite injectées dans le payload de l'événement de détection avant son envoi vers la file d'attente ou la base de données. Cette architecture modulaire assure une séparation des préoccupations et facilite la maintenance et l'évolution du système. L'enrichissement avec les données SatNOGS est donc une extension relativement directe qui confère au système une profondeur analytique considérable, passant d'une simple détection à une véritable intelligence opérationnelle de la mission.

Axe 4 : Conception d'une Architecture Microservice Optimisée pour la Latence et la Gestion de Charge

La mise en place d'un microservice dédié au prétraitement des images Sentinel-1, dans un environnement Docker léger et avec des contraintes de RAM raisonnables, impose des choix architecturaux stricts visant à optimiser la latence et à gérer les pics de charge. La faisabilité de cette architecture est jugée **modérée**, car elle repose sur une stratégie de traitement efficace pour surmonter les goulots d'étranglement inhérents au traitement d'images satellitaires volumineuses sur un processeur unique. L'approche la plus prometteuse est d'alléger drastiquement la charge de travail du microservice en ne traitant que les données nécessaires.

Le temps de traitement du prétraitement d'une scène Sentinel-1 GRD complète (environ 250km x 250km) sur un processeur central seul avec des bibliothèques Python pures comme `rasterio` et `numpy` serait prohibitif pour un flux quasi en temps réel. Les opérations de calibration, de filtrage et de normalisation sont gourmandes en calculs. Un temps de

traitement estimé se situe entre 5 et 15 minutes par scène, une latence inacceptable pour la surveillance maritime. La clé pour surmonter ce défi réside dans une stratégie de **réduction spatiale**. Plutôt que de télécharger et de traiter la scène entière, le microservice doit demander au serveur de données (idéalement Sentinel Hub) de fournir uniquement le sous-ensemble spatial de l'image qui correspond à la zone d'intérêt (par exemple, une zone côtière spécifique ou une zone de haute densité de trafic). La **Sentinel Hub Process API** est la technologie qui rend cette approche viable ¹. En définissant un script d'évaluation qui inclut une opération de découpe spatiale, il est possible de télécharger une image de quelques dizaines de mégaoctets au lieu de plusieurs gigaoctets. Cette réduction massive de la taille des données à traiter diminue proportionnellement le temps de lecture, de calcul et de normalisation, ramenant la latence du prétraitement à un niveau acceptable (quelques secondes).

Une fois la stratégie de téléchargement optimisée, l'architecture du système doit être conçue pour être scalable et résiliente. Une approche de microservices est parfaitement adaptée à ce besoin. Le système peut être décomposé en plusieurs services autonomes, chacun ayant une responsabilité unique ²⁵. Un service **data-ingestor** pourrait gérer la recherche des produits sur le CDSE et l'appel à la Process API de Sentinel Hub. Un service **preprocessor** recevrait les zones d'intérêt réduites, exécuterait la chaîne de traitement (calibration, speckle, normalisation, tiling) et produirait des tuiles d'images prêtes à être analysées. Un service **detector** exécuterait le modèle YOLOv8 sur ces tuiles. Enfin, un service **aggregator** collecterait les résultats, les enrichirait avec les données de télémétrie et de position satellite provenant de SatNOGS, et les stockerait dans une base de données. Cette décomposition permet de mettre à l'échelle chaque service indépendamment en fonction de ses besoins en ressources.

Pour gérer les pics de charge et assurer une distribution équilibrée de la charge, l'intégration d'une **file d'attente** et d'un **cache** est indispensable. Une file d'attente, comme Redis ou RabbitMQ, agirait comme une couche tampon entre les producteurs de tâches (le service **preprocessor**) et les consommateurs (le service **detector**). Le service **preprocessor** placerait les tuiles d'images dans la file d'attente, tandis que plusieurs instances du service **detector** (potentiellement sur différents hôtes) consommeraient ces tâches. Cette architecture décentralisée permet d'équilibrer la charge, de faire face à des pics de demande sans saturer le GPU/CPU, et d'améliorer la redondance. Parallèlement, un **cache** rapide comme Redis peut être utilisé pour stocker les tuiles d'images prétraitées les plus récemment traitées. Si une nouvelle détection est demandée pour une même zone très proche dans le temps, le service de détection peut immédiatement servir la tuile depuis le cache, évitant ainsi un cycle complet de

téléchargement et de prétraitement, ce qui réduit encore davantage la latence globale ⁶⁴
⁷⁴.

Le choix du format de données pour la communication entre ces microservices est également crucial pour la latence. Bien que les images soient souvent échangées au format PNG, ce format introduit une surcharge de compression/décompression et peut entraîner une perte de précision si une compression sans perte n'est pas utilisée. Le format natif NumPy (.npy) est une alternative beaucoup plus performante. Il conserve la matrice d'octets brute en format binaire, sans aucune surcharge de métadonnées, et peut être chargé en mémoire centrale par le modèle de détection (par exemple, ONNX Runtime) avec une latence minimale. Le format GeoTIFF pourrait également être envisagé, mais il est plus lourd et complexe à manipuler que nécessaire pour l'étape d'inférence. Le format .npy offre le meilleur compromis entre efficacité et simplicité pour le transport de données d'image entre les services.

En résumé, une architecture microservice orientée latence est réalisable en combinant trois éléments clés : 1) une stratégie de téléchargement de données réduites via Sentinel Hub Process API ; 2) une décomposition du pipeline en services autonomes communiquant via une file d'attente (Redis/RabbitMQ) et un cache (Redis) ; et 3) l'utilisation du format .npy pour la transmission des données d'image. Cette architecture permet de créer un système non seulement performant mais aussi modulaire, extensible et capable de s'adapter à une charge de travail variable.

Synthèse Stratégique et Feuille de Route Recommandée

L'intégration de données Sentinel-1 dans le pipeline maritime-edge-ai est un projet ambitieux qui transforme le démonstrateur technologique en un système de surveillance maritime opérationnel. Les analyses des quatre axes techniques révèlent que si les technologies de base sont matures et accessibles, leur intégration requiert une approche méthodique et une validation expérimentale rigoureuse. Le succès du projet dépend de la capacité à surmonter les défis de prétraitement SAR, à choisir la plateforme de données la plus adaptée, à enrichir les détections avec des données orbitales, et à architecturer un système léger et performant. La faisabilité générale du projet est évaluée comme **modérée**, car elle repose sur la validation de plusieurs points d'incertitude critiques, notamment la compatibilité distributionnelle entre les données Sentinel-1 et les modèles d'IA.

Le premier axe, le prétraitement des images Sentinel-1 GRD, est le plus critique. Il a été établi que la chaîne de traitement minimale doit impérativement inclure la calibration radiométrique en σ^0 et le filtrage du speckle pour obtenir des données exploitables par le modèle YOLOv8 [6](#) [85](#). Cependant, le point le plus incertain est la distribution d'intensité finale et sa compatibilité avec les données d'entraînement SSDD/iVision-MRSSD, pour lesquelles aucune information sur le prétraitement n'est disponible. Cette lacune nécessite une validation expérimentale prioritaire pour valider la chaîne de traitement et ajuster les paramètres de normalisation.

Le deuxième axe, la plateforme de données, a montré que la **Sentinel Hub Process API** est l'outil le plus puissant pour répondre aux contraintes de l'architecture. Sa capacité à traiter et à découper spatialement les données directement dans le nuage est la solution la plus efficace pour réduire la latence et la charge sur le microservice local [1](#). Bien que le CDSE reste une source de données brute fiable, son usage devrait être réservé à des besoins de repli ou d'analyse hors ligne, tandis que la Process API devient le pilier du flux de données en temps réel.

Le troisième axe, l'intégration de données SatNOGS, est une extension relativement facile mais hautement bénéfique. La récupération des TLE et de la télémétrie via les API publiques de SatNOGS permet d'enrichir chaque détection d'un contexte orbital et de santé du satellite, augmentant ainsi la fiabilité et la valeur analytique des résultats [67](#) [91](#). Cette intégration peut être réalisée par la création d'un microservice dédié qui communique avec les autres services via une API REST.

Enfin, le quatrième axe, l'architecture microservice, est réalisable en adoptant une stratégie de traitement allégé. Le prétraitement d'une scène complète sur CPU est trop lent. La solution passe par le téléchargement de zones d'intérêt réduites via la Process API de Sentinel Hub. Cette approche, combinée à une architecture décomposée en services et à l'utilisation d'une file d'attente (ex: RabbitMQ) et d'un cache (ex: Redis) pour la gestion de charge, permet de construire un système performant et évolutif [25](#) [64](#).

Compte tenu de ces constats, la feuille de route suivante est recommandée pour guider le projet jusqu'à son achèvement :

Phase 1 : Validation Expérimentale Fondamentale (P0 - Priorité Absolue) 1.

Objectif : Valider la chaîne de prétraitement et la compatibilité distributionnelle. 2.

Actions :

- Télécharger manuellement un petit jeu de test de 5 à 10 scènes Sentinel-1 GRD IW via l'API CDSE.

- Développer un prototype de script Python pur (`rasterio`, `numpy`) qui applique la chaîne de traitement complète : calibration, filtrage de speckle, normalisation et tiling.
- Créer un petit script de benchmarking pour mesurer le temps de traitement CPU total sur ce jeu de test.
- Appliquer ce prétraitement aux tuiles et les alimenter dans le modèle YOLOv8 existant. Mesurer la performance (Précision, Rappel, F1-score) et comparer avec les résultats de référence du PoC.

1. **Résultat Attendu** : Un rapport de validation confirmant la faisabilité du prétraitement et identifiant les paramètres optimaux pour la normalisation. Si la performance chute, itérer sur la chaîne de traitement.

Phase 2 : Mise en Place de l'Infrastructure de Flux de Données (P1) 1. **Objectif** : Mettre en place l'architecture de microservices et le pipeline de données. 2. **Actions** :

- Définir les interfaces de communication (REST/WebSocket) entre les microservices (`data-ingestor`, `preprocessor`, `detector`, `aggregator`).
- Implémenter le service `data-ingestor` qui utilise l'API OData du CDSE pour trouver les produits et, de manière cruciale, appelle la **Sentinel Hub Process API** pour télécharger les zones d'intérêt prétraitées.
- Déployer l'ensemble des services dans un environnement Docker avec Docker Compose.
- Intégrer une file d'attente Redis ou RabbitMQ pour la gestion de charge entre les services.

1. **Résultat Attendu** : Un pipeline de données fonctionnel qui prend une boîte englobante en entrée, récupère les données Sentinel-1 correspondantes, les prétraite et les envoie à un service de détection pour production.

Phase 3 : Intégration des Données SatNOGS (P1) 1. **Objectif** : Enrichir les détections avec des données orbitales et de télémétrie. 2. **Actions** :

- Créer un nouveau microservice `satellite-monitor` qui récupère périodiquement les TLE depuis l'API SatNOGS.
- Modifier le service `detector` pour qu'il appelle le service `satellite-monitor` afin d'obtenir la position du satellite et la télémétrie la plus proche temporellement pour chaque détection.

- Mettre à jour le schéma de la base de données pour stocker ces nouvelles informations (position, température, batterie, etc.).

1. **Résultat Attendu** : Des événements de détection enrichis avec un contexte orbital et de santé du satellite.

Phase 4 : Optimisation et Déploiement Final (P2) 1. **Objectif** : Optimiser les performances et préparer le déploiement. 2. **Actions** :

- Analyser les logs et les métriques pour identifier les goulots d'étranglement restants.
- Explorer des optimisations avancées, comme l'application de techniques d'optimisation au modèle YOLOv8 (quantification plus agressive, pruning) pour améliorer la latence sur le matériel cible 58 60 .
- Mettre en place un système de monitoring centralisé (ex: Prometheus/Grafana) pour surveiller la santé et la performance de l'ensemble du système.
- Documenter l'ensemble du système pour faciliter sa maintenance et son évolution.

1. **Résultat Attendu** : Un système de surveillance maritime robuste, performant, et prêt à être déployé dans un environnement de production.

En suivant cette feuille de route structurée, le projet peut progressivement surmonter les défis techniques identifiés et aboutir à un système opérationnel qui capitalise pleinement sur les données satellitaires réelles et les informations de télémétrie pour une surveillance maritime intelligente et contextuelle.

Référence

1. Sentinel-1 GRD - Documentation - Copernicus <https://documentation.dataspace.copernicus.eu/APIs/SentinelHub/Data/S1GRD.html>
2. [PDF] GMES Space Component Sentinel-3 Payload Data Ground ... <https://earth.esa.int/eogateway/documents/20142/1564943/Sentinel-3-PDGS-Products-Definition-Document.pdf>
3. Copernicus Data Space Ecosystem | Europe's eyes on Earth <https://dataspace.copernicus.eu/>

4. Harmonized Sentinel-2 MSI: MultiSpectral Instrument, Level-2A (SR) https://developers.google.com/earth-engine/datasets/catalog/COPERNICUS_S2_SR_HARMONIZED
5. [PDF] Release 0.36 the pyroSAR Developers https://pyrosar.readthedocs.io/_/downloads/en/latest/pdf/
6. [PDF] ESA CCI HR https://climate.esa.int/media/documents/CCI_HRLC_Ph1-D2.2_ATBD_v4.0.pdf
7. Comparison of Long Short-Term Memory Networks and Random ... <https://www.mdpi.com/2072-4292/13/24/5000>
8. Landsat 8 | U.S. Geological Survey - USGS.gov <https://www.usgs.gov/landsat-missions/landsat-8>
9. Earth Observation Cloud Platform Concept Development Study Report <https://docs.ogc.org/per/21-023.html>
10. [PDF] Sen2Cor v 2.12.03: Installation, Configuration and Processing https://step.esa.int/thirdparties/sen2cor/2.12.0/docs/Sen2Cor_Quick_Guide_V_2_1_0.pdf
11. 11.0 Ground Data Systems and Mission Operations - NASA <https://www.nasa.gov/smallsat-institute/sst-soa/ground-data-systems-and-mission-operations/>
12. [PDF] Latin American CubeSat Workshop: - UFSC <https://cubesat.ufsc.br/2024/cb1.pdf>
13. archives - RTL-SDR.com <https://www.rtl-sdr.com/archives/>
14. Ship Detection with Deep Learning in Optical Remote-Sensing Images <https://www.mdpi.com/2072-4292/16/7/1145>
15. [PDF] Maritime Surveillance Finding Dark Ships with Satellites and ... https://backend.orbit.dtu.dk/ws/files/390125979/Maritime_Surveillance_Finding_Dark_Ships_with_Satellites_and_Artificial_Intelligence.pdf
16. satellite-image-deep-learning/techniques - GitHub <https://github.com/satellite-image-deep-learning/techniques>
17. PAZ SAR Satellite Mission of Spain - eoPortal <https://www.eoportal.org/satellite-missions/paz>
18. [PDF] Marine Environmental Impact on CFAR Ship Detection as Measured ... <http://mtc-m21d.sid.inpe.br/col/sid.inpe.br/mtc-m21d/2023/07.24.12.26/doc/remotesensing-15-03441.pdf>
19. Automated Collection Planning for Civilian and Commercial Satellite ... <https://ieeexplore.ieee.org/iel8/4609443/4609444/10945445.pdf>
20. S1 Mission - SentiWiki - Copernicus <https://sentiwiki.copernicus.eu/web/s1-mission>

21. Application of Multitemporal Change Detection in Radar Satellite ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC10223237/>
22. sacridini/Awesome-Geospatial - GitHub <https://github.com/sacridini/Awesome-Geospatial>
23. [PDF] Notes on Synthetic Aperture Radar Image Quality - OSTI <https://www.osti.gov/servlets/purl/2516837>
24. fadeoutsoftware/WASDI: Web Advanced Space Developers Interface <https://github.com/fadeoutsoftware/WASDI>
25. Talks - FOSS4G Europe 2025 <https://2025.europe.foss4g.org/schedule/talks/>
26. [PDF] NASA High-End Computing Program User Needs Assessment 2020 ... https://hec.nasa.gov/workshop20/HEC_Needs_Assessment_2020_AppD_c.pdf
27. Courses - Learning Tree <https://www.learningtree.co.uk/courses/>
28. Sensors, Volume 22, Issue 17 (September-1 2022) – 401 articles <https://www.mdpi.com/1424-8220/22/17>
29. [XLS] Technology Skills - O*NET Resource Center https://www.onetcenter.org/dl_files/frameworks/Technology_Skills_Competencies.xlsx
30. BAT Bangladesh is hiring #Position: Analyst- IDT Operations Dhaka ... <https://www.facebook.com/bhawmj3821/posts/bat-bangladesh-is-hiring-position-analyst-idt-operations-dhaka-dhaka-bangladesh-/122163074330875821/>
31. Character-BERT - Hugging Face https://huggingface.co/helboukkouri/character-bert/raw/main/mlm_vocab.txt
32. job listing - Coolbuffs https://coolbuffs.com/jobs?job_skill_id%5B%5D=15107
33. Chen Tian: Publications <https://cs.nju.edu.cn/tianchen/publications.html>
34. Call For Papers - IEEE Communications Society <https://www.comsoc.org/publications/journals/ieee-tnse/cfp>
35. Computer Science - arXiv <https://arxiv.org/list/cs/new>
36. IEEE Special Issue Tracker <https://klb2.github.io/ieee-special-issue-tracker/>
37. Research and Advances - Communications of the ACM <https://cacm.acm.org/section/research/>
38. PUBLICATIONS - Networked Computing Lab. <https://nxc.snu.ac.kr/publications>
39. Selected Publications <http://ansrlab.cse.cuhk.edu.hk/publications/>
40. Best paper awards - ACM SIGCOMM 2025 <https://conferences.sigcomm.org/sigcomm/2025/program/papers-info/>
41. Publications - Abusayeed Saifullah <https://www.abusayeedsaifullah.com/publications>

42. Publications - Prof. Danda B. Rawat <https://www.rawatonline.com/research/publications>
43. S2 Products - SentiWiki - Copernicus <https://sentiwiki.copernicus.eu/web/s2-products>
44. STAC product catalogue - Documentation <https://documentation.dataspace.copernicus.eu/APIs/STAC.html>
45. [PDF] ITALIAN SPACE INDUSTRY - Agenzia Spaziale Italiana https://www.asi.it/wp-content/uploads/2020/07/CatalogoASI_2018_web_rid.pdf
46. [XLS] mjc - DDTC https://www.pmdtc.state.gov/sys_attachment.do?sys_id=eb4ac16247157a9027972464336d4351
47. [PDF] Earth Observation: - Data, Processing and Applications <https://eopublications.au/content/support/Vol2D%20144%2020251007.pdf>
48. User guide — SatNOGS Client 2.1.1 documentation <https://docs.satnogs.org/projects/satnogs-client/en/stable/userguide.html>
49. [PDF] Range Geolocation Accuracy of C-/L-Band SAR and its Implications ... https://web.gps.caltech.edu/~simons/publications/pdfs/Zhang_etal_2022.pdf
50. [PDF] prospects for high accuracy time dissemination & synchronization ... <https://apps.dtic.mil/sti/tr/pdf/ADA511249.pdf>
51. Measurements on the Absolute 2-D and 3-D Localization Accuracy ... <https://www.mdpi.com/2072-4292/10/4/656>
52. [PDF] Delivery Note - ESA Earth Online https://earth.esa.int/eogateway/ftp/Sample-data/eGeos/eGeos_SET1/CSG_SCANSAR-2/Level%20SCS_B%20hdf5%20HV/DFDN_CSG_SSAR2_SCS_B_0101_SC2_003_HV_RA_F_20220822043643_20220822043712_1_F_53N_Z33_N00.h5.pdf
53. Signal Processing and Waveform Re-Tracking for SAR Altimeters on ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC10674187/>
54. [PDF] Analysis and Optimization of MirrorSAR Synchronization https://elib.dlr.de/215046/1/ustalli_et_ali_EUSAR2024_final.pdf
55. Chapter VI SATELLITE GEODESY - National Geodetic Survey (NGS) https://geodesy.noaa.gov/PUBS_LIB/Geodesy4Layman/TR80003D.HTM
56. Sentinel-1 Extended Time Annotation Products (ETAD) <https://browser.stac.dataspace.copernicus.eu/collections/sentinel-1-etad>
57. CYGNSS Level 3 Science Data Record Version 3.1 - PO.DAAC https://podaac.jpl.nasa.gov/dataset/CYGNSS_L3_V3.1
58. Edge-Optimized Lightweight YOLO for Real-Time SAR Object ... <https://www.mdpi.com/2072-4292/17/13/2168>
59. Benchmarking Deep Learning Models for Object Detection on Edge ... <https://arxiv.org/html/2409.16808v1>

60. Publications | Borui LI 李博睿 <https://www.liborui.cn/publication/>
61. [PDF] Accelerated Edge Computing Platforms for Object Detection Model ... <https://www.diva-portal.org/smash/get/diva2:1987606/FULLTEXT01.pdf>
62. Benchmarking YOLOv8 Variants for Object Detection Efficiency on ... https://www.researchgate.net/publication/400413845_Benchmarking_YOLOv8_Variants_for_Object_Detection_Efficiency_on_Jetson_Orin_NX_for_Edge_Computing_Applications
63. Publications | Meng Li's Homepage <https://mengli.me/publication/>
64. Papers <https://www.ieee-ai-for-science.org/2024/ispa/ispa-2024-accepted.html>
65. Benchmarking YOLOv8 Variants for Object Detection Efficiency on ... <https://www.mdpi.com/2073-431X/15/2/74>
66. Publications - Yang (Henry) Liu <https://yanghenryliu.github.io/publications/>
67. [PDF] Release 0+untagged.1.g28c7e1b SatNOGS https://acinonyx-satnogs-client.readthedocs.io/_/downloads/en/latest/pdf/
68. [PDF] PRIMARY STUDY OF GEOMETRIC ACCURACY FOR SAR ... <https://acrs-aars.org/proceeding/ACRS2023/Poster%20I/ACRS2023068.pdf>
69. [PDF] S1-ETAD - SentiWiki https://sentiwiki.copernicus.eu/__attachments/1673968/ETAD-DLR-PS-0014%20-%20Sentinel-1%20Extended%20Timing%20Annotation%20%28ETAD%29%20-%20Product%20Specification%20Document%20-%201.9.pdf
70. Doppler Bandwidth Estimation for Short-Range SAR Considering ... https://www.researchgate.net/publication/396299415_Doppler_Bandwidth_Estimation_for_Short-Range_SAR_Considering_Signal_Attenuation
71. Structure and Performance Analysis of Signal Acquisition ... - MDPI <https://www.mdpi.com/1424-8220/21/2/525>
72. [PDF] An Algorithm for Harsh Doppler Shift Estimation for Satellite ... https://research.chalmers.se/publication/546045/file/546045_Fulltext.pdf
73. Introduction to the HERE HD GNSS Positioning API <https://docs.here.com/positioning/docs/readme-guide>
74. [PDF] A Survey of Edge Caching: Key Issues and Challenges - SciOpen https://www.sciopen.com/local/article_pdf/10.26599/TST.2023.9010051.pdf
75. Main Technical Program - IEEE INFOCOM 2026 <https://infocom2026.ieee-infocom.org/main-technical-program>
76. Xu Chen - Google Scholar <https://scholar.google.com/citations?user=BmiiIaoAAAAJ&hl=en>

77. Selected publications by topic (updated: February 2026) - Jun Zhang <https://eejzhang.people.ust.hk/publicationstopic.html>
78. Papers - Zili Meng <https://zilimeng.com/papers.html>
79. [PDF] Overload: Latency Attacks on Object Detection for Edge Devices https://openaccess.thecvf.com/content/CVPR2024/papers/Chen_Overload_Latency_Attacks_on_Object_Detection_for_Edge_Devices_CVPR_2024_paper.pdf?ref=blog.unitlab.ai
80. Geometric Correction of Side-Scan Sonar Images with Image ... - arXiv <https://arxiv.org/html/2604.19901>
81. An Adaptive Ship Detection Scheme for Spaceborne SAR Imagery <https://pmc.ncbi.nlm.nih.gov/articles/PMC5038623/>
82. GitHub - shashwatak/satellite-js: Modular set of functions for SGP4 ... <https://github.com/shashwatak/satellite-js>
83. High-Speed Lightweight Ship Detection Algorithm Based on YOLO ... <https://www.mdpi.com/2072-4292/13/10/1909>
84. jasonmanesis/Ship-Detection-on-Remote-Sensing-Synthetic ... <https://github.com/jasonmanesis/Ship-Detection-on-Remote-Sensing-Synthetic-Aperture-Radar-Data>
85. Optimizing toward efficiency for SAR image ship detection <https://www.climatechange.ai/papers/neurips2022/5>
86. [PDF] Efficient Size and Heading Angle Estimation of Ships in SAR and ... https://elib.dlr.de/204142/1/Efficient_Size_and_Heading_Angle_Estimation_of_Ships_in_SAR_and_ISAR_Images.pdf
87. [PDF] A Novel Algorithm for Ship Detection in SAR Imagery Based on the ... <https://upcommons.upc.edu/bitstreams/d26406d3-c6be-4db2-9815-1871bab708d2/download>
88. Rc-W024/SAR_Ship_detection_CFAR: 🚢 Official ... - GitHub https://github.com/Rc-W024/SAR_Ship_detection_CFAR
89. Research on spaceborne SAR image preprocessing based on ... - DOI <https://doi.org/10.1080/2150704X.2026.2661264>
90. [DOC] R4-2513627 draft big CR for 38.101-5 for NTN NGSO ... - 3GPP https://www.3gpp.org/ftp/tsg_ran/wg4_radio/TSGR4_116bis/Inbox/Post_Meeting/%5Bpost%20116bis%5D%5B328%5D%20Rel-19%20Demodulation_Part3/R4-2513627%20draft%20big%20CR%20for%2038.101-5%20for%20NTN%20NGSO%20testing_V2_Samsung.docx
91. Question about api/telemetry - Software - Libre Space Community <https://community.libre.space/t/question-about-api-telemetry/6938>

92. User guide — SatNOGS Client 1.7+0.g0b8b7ec.dirty documentation <https://docs.satnogs.org/projects/satnogs-client/en/1.7/userguide.html>
93. <http://arewemeetingyet.com/UTC/2017-07-17/18:00/Sa...> <http://arewemeetingyet.com/UTC/2017-07-17/18:00/SatNOGS-HW-Docs>
94. Overview - Documentation Center <https://developers.satellologic.com/index.html>
95. HERE HD GNSS Positioning API Terminology <https://docs.here.com/positioning/docs/terminology>
96. GPS-Based Time & Phase Synchronization Processing for ... https://www.researchgate.net/publication/224594852_GPS-Based_Time_Phase_Synchronization_Processing_for_Distributed_SAR
97. Sentinel-1 - Documentation - Copernicus <https://documentation.dataspace.copernicus.eu/Data/SentinelMissions/Sentinel1.html>
98. Sentinel-1 GRD <https://docs.sentinel-hub.com/api/latest/data/sentinel-1-grd/>
99. Missing Sentinel-1 RESORB after March 2 2026 - STEP Forum <https://forum.step.esa.int/t/missing-sentinel-1-resorb-after-march-2-2026/45594>
100. Sentinel 1 GRD - Planet Documentation <https://docs.planet.com/data/public-data/copernicus/sentinel-1-grd/>
101. Sentinel 1 Level-1 Ground Range Detected (GRD) <https://planetarycomputer.microsoft.com/dataset/sentinel-1-grd>
102. Search for available data — Sentinel Hub 3.11.5 documentation https://sentinelhub-py.readthedocs.io/en/latest/examples/data_search_cdse.html
103. Sentinel-1 Precise Orbit Determination (POD) Products <https://registry.opendata.aws/s1-orbits/>
104. CSC – Sentinel-2 Product Unit Definition and Metadata ICD – EOF <https://eof.esa.int/document/esa-eo-framework-eof-csc-sentinel-2-product-unit-definition-and-metadata-icd/>
105. SENTINEL-1 Level 1 Ground Range Detected (GRD) https://data.destination-earth.eu/data-portfolio/EO.ESA.DAT.SENTINEL-1.L1_GRD
106. 2. Pre-processing and preparation of input Sentinel-1 and ancillary ... <https://extwiki.eodc.eu/GFM/PDD/Preprocessing>
107. Research on GEO SA-Bi SAR Imaging based on Joint Radar ... <https://arxiv.org/html/2508.12890>
108. An Adaptive Ship Detection Scheme for Spaceborne SAR Imagery https://www.researchgate.net/publication/307110948_An_Adaptive_Ship_Detection_Scheme_for_Spaceborne_SAR_Imagery
109. 基于多特征加权的SAR影像舰船检测优化方法 - 通信学报 <https://www.joconline.com.cn/zh/article/doi/10.11959/j.issn.1000-436x.2020013/>

110. bopen/sarsen: Algorithms and utilities for Synthetic ... - GitHub <https://github.com/bopen/sarsen>
111. 中南大学 xubing <https://faculty.csu.edu.cn/xubing/en/lwcg/61776/content/44008.htm>